

Lab sheet 10**Submission Due: 15.07.2024****Title:** Web Security in PHP and JavaScript**Aims:**

- Implement security measures in PHP and JavaScript

Tasks:

- Identify common security threats
- Implement PHP security practices
- Implement JavaScript security practices

Activities :

1. Identify the common security threats
 - I. SQL Injection
 - SQL Injection is a type of attack where an attacker can execute arbitrary SQL code on a database by manipulating user input.
 - This can lead to unauthorized access to data, modification of data, and even deletion of the database.
 - II. Cross-Site Scripting (XSS)
 - XSS is an attack where an attacker injects malicious scripts into content from otherwise trusted websites.
 - This leads to data theft, session hijacking, and other malicious actions.
 - III. Cross-Site Request (CSRF)
 - CSRF is an attack that tricks the user into submitting a malicious request.
 - It exploits the trust a web application has in the authenticated user.
 - IV. Remote Code Execution (RCE)
 - RCE allows an attacker to execute arbitrary code on the server, often leading to complete server compromise.
2. Implement PHP security practices
 - I. Input validation and sanitization

```
$username = filter_var($data->username, FILTER_SANITIZE_STRING);  
$password = filter_var($data->password, FILTER_SANITIZE_STRING);  
  
$user->setUsername($username);  
$user->setPassword($password);
```

II. SQL injection prevention

```
public function create()
{
    $query = "INSERT INTO `expenses` (`description`, `amount`) VALUES
(?, ?)";
    $stmt = $this->connection->prepare($query);

    if ($stmt === false) {
        return false;
    }

    $stmt->bind_param('sd', $this->description, $this->amount);

    $result = $stmt->execute();

    $stmt->close();

    return $result;
}
```

III. XSS prevention

```
$username = htmlspecialchars($data->username, ENT_QUOTES, 'UTF-8');
$password = htmlspecialchars($data->password, ENT_QUOTES, 'UTF-8');
$email = htmlspecialchars($data->email, ENT_QUOTES, 'UTF-8');

$user->setUsername($username);
$user->setPassword($password);
$user->setEmail($email);
```

IV. Session management

```
<?php
$cookieParams = [
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
];
```

```
session_set_cookie_params($cookieParams);

session_start();

require_once '../database/User.php';

$databse = new Database("localhost", "root", "", "expense_tracker");
$db = $databse->getConnection();

$user = new User($db);

$username = htmlspecialchars($data->username, ENT_QUOTES, 'UTF-8');
$password = htmlspecialchars($data->password, ENT_QUOTES, 'UTF-8');

$user->setUsername($username);
$user->setPassword($password);

if ($user->login()) {
    session_regenerate_id(true);

    $_SESSION['username'] = $user->username;

    echo json_encode(["message" => "Login successful."]);
} else {
    echo json_encode(["message" => "Login failed."]);
}
?>
```

3. Implement JavaScript security practices

- I. Avoid inline JavaScript
- II. Content Security Policy (CSP)

```
<head>
  <title>User Authentication</title>
  <meta http-equiv="Content-Security-Policy"
        content="default-src 'self'; script-src 'self'; style-src 'self';
img-src 'self';">
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
```

III. DOM manipulation

```
async function register() {
    const username =
document.getElementById('registerUsername').value;
    const password =
document.getElementById('registerPassword').value;
    const email = document.getElementById('registerEmail').value;
```

```
try {
  const response = await fetch('php/register.php', {
    method: 'POST',
    body: JSON.stringify({ username, password, email }),
    headers: { 'Content-Type': 'application/json' }
  });

  if (!response.ok) {
    throw new Error('Network response was not ok');
  }

  const result = await response.json();

  if (result.success) {
    showLoginForm();
    alert('Registration successful. Please login.');
```

```
  } else {
    alert('Registration failed: ' + result.message);
  }
} catch (error) {
  alert('Error registering user: ' + error.message);
}
```

Discussion:

- CSRF Prevention
- How to handle CORS on the server side using PHP